

Autonomous Knowledge Execution Agent

Design write-up | Assessment Option 3 | Utsav Chopra

Code: <https://github.com/Utsavchopragithub/Autonomous-Knowledge-Execution-Agent>

1. Domain and goal

I chose Option 3 (Autonomous Knowledge Execution Agent) and built it for an **internal IT / HR helpdesk**. An employee types a request in plain language. The agent retrieves facts from several internal knowledge sources, reasons about what is needed, decides which action(s) to take, asks a human before anything critical or irreversible, executes the action(s), and explains its reasoning before and after execution. Every step is written to an audit log.

2. Architecture in brief

The agent is a LangGraph state machine: **retrieve -> recall -> plan -> (escalate | execute) -> explain**. The architecture diagram is submitted as a separate file.

- **Four knowledge sources, searched together:** policy JSON, ticket-history CSV, a SQLite employee database, and an unstructured handbook (plain text, split into overlapping chunks). All are normalised into one document format and merged into one hybrid index (FAISS semantic search + BM25 keyword search); each result carries its source label.
- **Long-term memory:** every past step in the audit log is embedded and searched by meaning, so a relevant interaction can be recalled even if it is many turns old, not only the last few.
- **Planning:** the LLM (Gemini free tier) returns JSON containing a confidence level, a description of any conflict or gap, and an ordered list of steps, each with its own reasoning and a *depends_on* list.
- **Execution:** steps are grouped into dependency batches. Independent, non-critical steps in a batch run in parallel (ThreadPoolExecutor). Critical steps are taken out of the batch and run one at a time, each blocked on explicit human approval.
- **Audit trail:** every step (parallel or not, approved or rejected) is logged to SQLite under a shared *plan_id*.

3. Key design decisions and why

Decision	Reasoning
The LLM decides; code only enforces	No canned answers or if/else routing on query text. Python retrieves data, validates the plan, runs approved actions, and logs. This follows the brief's requirement for autonomous LLM reasoning.
Escalate instead of guess	If confidence is low or sources disagree, the graph routes to an HR escalation rather than picking a number. The policy JSON says 18 leave days and the handbook says about 20; this conflict is deliberate test data for this behaviour.
Critical actions never run inside a parallel batch	A person must read and approve each irreversible action (password reset, account deactivation), so those run singly and synchronously.
Hybrid retrieval (semantic + keyword)	Semantic search handles paraphrases; BM25 catches exact terms and IDs. Combined 60/40 with min-max normalisation.
Fail safe on malformed LLM output	If the plan JSON cannot be parsed, the agent falls back to escalation with low confidence instead of crashing or acting on garbage.
Free, local-first stack	Gemini free tier, sentence-transformers, FAISS, rank_bm25, LangGraph and SQLite. No paid service is required to run the project.

4. Assumptions and mock data

- **All data is synthetic.** The policy entries, the 7 past tickets, the 4 employee records and the handbook excerpt were written by me for this assessment; no real company data is used.
- **Actions are simulated.** `create_ticket`, `request_leave`, `reset_password`, `escalate_to_hr` and similar functions return structured results; there is no real ticketing system or HRMS behind them. Leave balance and similar-ticket lookups do read the real SQLite database and CSV file.
- The employee ID is entered at the start of the session and trusted as given (no authentication).
- The 18 vs about 20 leave-day conflict between the JSON and the handbook is intentional, to demonstrate conflict handling.

5. Testing performed

I tested the system manually through the CLI with scenario queries and checked the behaviour and the audit log. I did not build an automated test suite or a quantitative benchmark, so I do not report accuracy figures.

Scenario	Behaviour checked
Slow laptop	Single step; similar past tickets drawn from the CSV history
Leave balance for a given employee ID	Value read from the SQLite database, not generated
Balance check + similar tickets	Two independent steps executed in parallel
Check balance, then request leave	Dependent step executed after its dependency
Total leave days per year	Conflict between sources detected; escalated to HR
Password reset	Approval prompt shown; rejection blocks the action, approval runs it

6. Limitations

- Conflict and confidence detection depends on the LLM's judgement over the retrieved snippets. It is not guaranteed to flag every contradiction and can occasionally over-escalate.
- Dependent steps run in the right order, but a step's output is not fed into the next step's parameters, so true conditional branching (for example, request leave only if the balance is above N) is not implemented.
- The parallel speed-up is illustrative: the simulated actions are fast. It would matter with real network-bound actions.
- Long-term memory is rebuilt from the audit log at start-up rather than updated incrementally; a large history would need a persisted index.
- Human approval is a command-line prompt only, and there is no authentication or role check on who is asking.
- The knowledge base is small (24 documents). Retrieval thresholds were set by hand and have not been tuned on a larger corpus.
- The free Gemini tier has rate limits, and model names change over time; the model is configurable through the `GEMINI_MODEL` environment variable.

7. How I built it

I used an AI assistant (Claude) as a pair-programming aid for design discussion and for drafting code. I chose the option and use case, set up the environment, ran and debugged the system on my machine, and tested the scenarios above.